Penetration Testing

LULAMELA MAXAKATHO
STUDENT NO: 602898

23 MARCH 2026

MILESTONE 1

ARCHITYPE VAMABONDSBE VANDE

Contents

Introduction 3

Environment Setup 3

Creating a Safe Test Environment 3

Create test folder: 3

Add fake system files: 4

Writing the Simulated Malware Script 4

Explanation of Code 5

Make script executable: 5

Running the Script 5

Documenting Results 6

Script Running Showing before, And after the attack. 6

Proof of simulation: 6

Conclusion 6

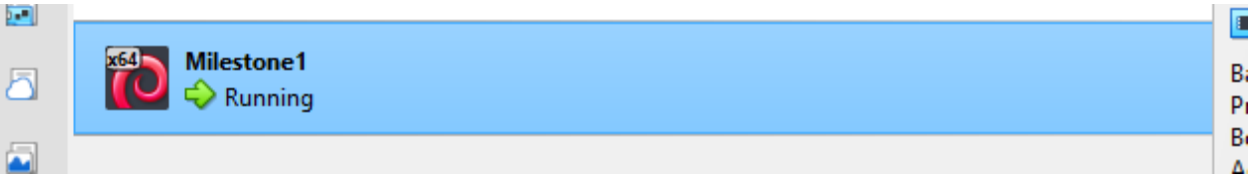
References 8

Introduction

In modern society, cybersecurity is crucial for protecting sensitive information and maintaining system integrity. This project simulates a destructive malware attack in a safe and controlled environment. The objective is to understand how malware executes, targets files, and impacts a system without compromising the operating system itself.

Environment Setup

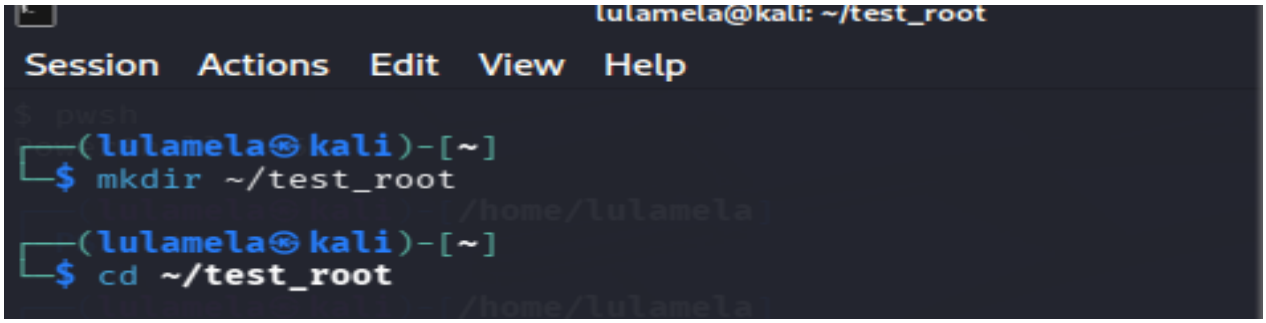
- A Kali Linux virtual machine was created inside VirtualBox.



Creating a Safe Test Environment

Create test folder:

- The cd ~ command was entered to navigate to the home directory.
- To create a directory or folder, the command mkdir was entered followed by the name of the folder, in this case, “test_root”.



- The purpose was to create a controlled environment to simulate the attack safely without damaging the actual operating system.

Add fake system files:

- A fake file was created in the test folder to safely simulate a malware attack. This allowed us to demonstrate file corruption and permission changes without risking real system files or the operating system itself, ensuring the experiment was controlled and safe.
- The command touch, was used to create the empty file to simulate critical system files.

```
┌──(lulamela㉿kali)-[~/test_root]
└─$ touch system.conf users.db boot.log data.txt
```

Writing the Simulated Malware Script

- A text editor on Kali Linux called Nano was opened using the command nano, followed by the name of the script “virus.sh”.
- The following code was pasted onto nano:

```
lulamela@kali: ~/test_root
Session  Actions  Edit  View  Help
  GNU nano 8.6                                   virus.sh
#!/bin/bash

# ──────────────────────────────────────────
# SIMULATED MALWARE SCRIPT
# ──────────────────────────────────────────

echo "Starting malicious program ... "

# Step 1: Move to fake root directory
cd ~/test_root

# Step 2: Show files before attack
echo "Files before attack:"
ls

# Step 3: Simulate destruction
echo "Simulating file corruption ... "

for file in *; do
    mv "$file" "$file.broken"
done

# Step 4: Remove permissions (simulate deeper damage)
chmod 000 *

# Step 5: Show files after attack
echo "Files after attack:"
ls

echo "System corruption simulation complete."

^G Help   ^O Write Out   ^F Where Is   ^K Cut   ^T Execute   ^C Location   M-U Undo   M-A Set Mark   M-]
```

Explanation of Code

- #!/bin/bash - Use Bash to run script
- echo - Display messages
- cd - Move to target directory
- ls - List files before and after simulation
- for loop - Iterate through files and rename them to .broken (simulating corruption)
- chmod 000 * - Remove read/write/execute permissions (simulating total system damage)

3. Save and exit:

- CTRL + X - Y - ENTER

Make script executable:

- The chmod command is used to change the permissions of files and directories in Linux. In this simulation, it was used to remove all read, write, and execute permissions from the fake files, which mimics how malware can lock or restrict access to critical files. This demonstrates the effect of an attack without actually harming the system.

```
┌──(lulamela㉿kali)-[~/test_root]
└─$ chmod +x virus.sh
```

Running the Script

- The script is ran using ./virus.sh. The script executes automatically.
- This demonstrates the simulated malware attack without touching real system files.

```
┌──(lulamela㉿kali)-[~/test_root]
└─$ ./virus.sh
```

- ./virus.sh tells Linux to execute the script file named virus.sh located in the current directory.
- The script lists the files in the test folder before the attack. It then renames each file to simulate corruption.
- It removes all file permissions to mimic a malware attack.

Documenting Results

Script Running Showing before, And after the attack.

```
┌──(lulamela㉿kali)-[~/test_root]
└─$ ./virus.sh
Starting malicious program...
Files before attack:
boot.log  data.txt  system.conf  users.db  virus.sh
Simulating file corruption...
Files after attack:
boot.log.broken  data.txt.broken  system.conf.broken  users.db.broken  virus.sh.broken
System corruption simulation complete.
```

- Before the attack, the system files do not have .broken in the end, however, after the attack, the system files are renames with .broken at the end.

Proof of simulation:

```
┌──(lulamela㉿kali)-[~/test_root]
└─$ ls
boot.log.broken  data.txt.broken  system.conf.broken  users.db.broken  virus.sh.broken
```

- The naming of the files with .broken at the end shows a simulation of a non-functional system.

Conclusion

This project successfully demonstrated a simulated malware attack in a controlled environment using a test folder in Kali Linux. By performing the simulation, we were able to safely illustrate how malware can affect files, including renaming them and changing permissions. The exercise

highlights the importance of safe testing practices, understanding malware behavior, and using Linux commands to simulate attacks without harming the actual operating system.

References

Linux Documentation Project (2020) *chmod Command.* Available at: https://tldp.org/LDP/abs/html/chmod.html (Accessed: 23 March 2026).

Offensive Security (2023) *Kali Linux Documentation.* Available at: https://www.kali.org/docs (Accessed: 23 March 2026).

Skoudis, E. and Liston, T. (2006) *Counter Hack Reloaded: A Step-by-Step Guide to Computer Attacks and Effective Defenses.* Upper Saddle River: Prentice Hall.